ECE 539 · Fall 2026 · Integration Project Proposal

Tumor Detection and Classification Using MRI Scans

Team Nxgen Diagnostics

Purpose and Data

Question: A neuroradiologist needs a fast first read on brain MRI scans. Our first starter model will take a single 2D brain MRI slice as input and output whether a tumor is present or not. A second model would also output the tumor type (glioma, meningioma, or pituitary). A key observation from our brief is that our dataset BRISC was derived from another dataset with no patient IDs and significant data leakage. Expert physicians de-duplicated and cleaned the images, but they could not guarantee that no patient appears in both training and test data, so some uncertainty remains in our evaluation.

Data: BRISC 2025 https://www.kaggle.com/datasets/briscdataset/brisc2025/ (Kaggle: briscdataset/brisc2025; Fateh et al., *Scientific Data* 2026).

Starter subset: Original dataset is 5,000 train / 1,000 test images. A smaller starter subset could be used for initial implementation debugging, e.g. 100 tumor and 100 not-tumor examples for the binary classifier; and for the multiclass classifier 100 examples per class as a starter subset adding up to 400. **Inputs:** 2D MRI slices (JPG) from axial, coronal, and sagittal planes, roughly balanced across planes. **Targets:** The binary classifier has labels YES tumor and NO tumor; the multiclass classifier distinguishes between four labels: (G) glioma, (M) meningioma, (P) pituitary tumor, or (N) no tumor. The four groups are about the same size. **Mapping:** The label comes from the folder the image is in (e.g., everything in the glioma/ folder is labeled glioma). We turn each label into a number for the model: glioma = 0, meningioma = 1, pituitary = 2, no tumor = 3. **Readiness** Verified access and free download; **Preprocessing:** resize images to 224x224; scale pixel values from 0 to 1; RGB format. **Split:** We keep the provided 1,000-image test set untouched. We take 15% of the 5,000 training images (750) as a validation set, keeping the four classes balanced. That leaves 4,250 for training. **Split unit:** Image. The dataset has no patient-level IDs, so we cannot guarantee patient independence in splits. There is a chance that slices from the same patient may appear in more than one split, which could make scores look slightly better than they really are, but medical experts revised it thoroughly to minimize this risk.

Research Basis and Architecture

Fateh *et al.* (2026a) [1] established benchmarks by technical validation using standard baseline models for comparing performance between different pretrained backbones. Among these the best performers were the EfficientNet, ResNet, and MobileNet families. Their results suggest this data performs best with EfficientNetB0, "which combines high accuracy with remarkable stability across all tumor types." [1] A limitation is that we could not find the exact architecture used for these baseline comparisons, as the paper states they used the backbones but did not specify the final classifying layer or details of their approach. We plan to add an MLP to the backbone consisting of the following sequence: a linear layer of 100 hidden units, a ReLU activation, a softmax output layer aligned with the number of target classes (2 for binary model; 4 for multiclass model), and an Adam optimizer with learning rate at about 10^{-3}, as this MLP was used in [2] with similar data and yielded results comparative to the benchmarks in [1]. Another potential approach is using a vision transformer, particularly the self-supervised DINO-family from Meta AI for general-purpose feature extractions as suggested in Missaoui *et al.* (2026). [2] They designed an effective classifier using DINO frozen backbones, a feature transformation with a scaler and PCA and feature fusion, and the final classifier (either an MLP or SVM). We were contemplating adapting their approach, but using the BRISC dataset instead of the Brain Tumor MRI Dataset, as well as a different augmentation strategy using tf.keras.layers since their “data augmentation methods ... are specific to each dataset.” Bolder approaches attempted to use hierarchical multi-task transformers like Swin-HAFNet for robust, dual-task analysis, but a limitation is that this is a more complex approach using concepts not discussed in class. [7]

Training

We will train an ImageNet-pretrained EfficientNet-B0 on the BRISC dataset for the binary and four-class classification tasks. The input will be a 224 × 224 RGB MRI image, and the target will be its encoded class label. Initially, the pretrained backbone will be frozen while the newly initialized (default settings) classification head with 100 units is trained (Fig. 1); if time permits, we will also evaluate partial fine-tuning of the final backbone blocks. The models will use two or four output logits, with BCE

and CE respectively, learning with the Adam optimizer. The initial learning rate will be approximately 0.001 for head training, with a smaller learning rate considered for backbone fine-tuning. The main training experiment will compare models trained with and without augmentation, using the same training/validation split and other settings.

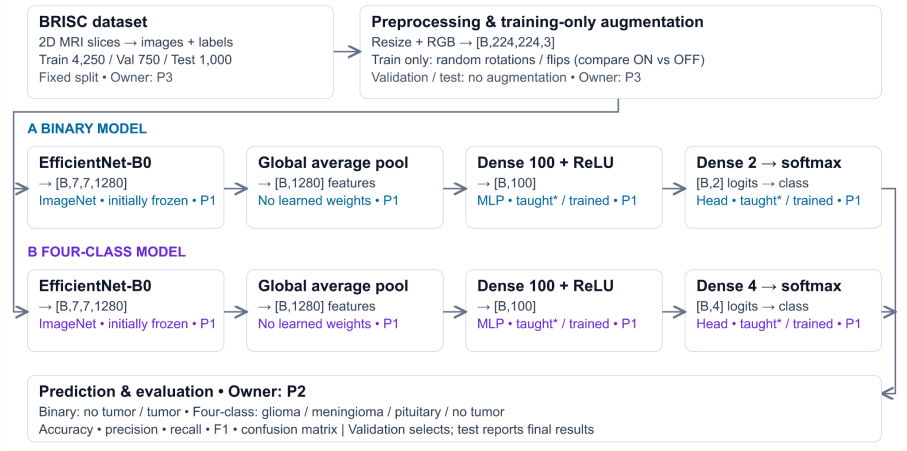


Figure 1. Proposed MRI tumor classification architecture created with OpenAI Codex [8]

Evaluation

Baseline: Our baseline will use a frozen EfficientNet pretrained on ImageNet for both the binary and multiclass models. We will train on BRISC, following the open-source implementation by @abdallahwagih. [3] **Justification**: it is a standard, well-documented approach for this exact task, written in TensorFlow (which we are using). His original results used Version 1 of BrainTumorMRIDataset, which has known data leakage, so we will retrain the model on BRISC to get a more reliable result. **Controlled comparison:** We will train the same EfficientNet twice: once with data augmentation (small rotations and flips on the training images) and once without. All other variables will remain constant: same source data, split, image dimensions, and training settings. We compare validation accuracy to see whether augmentation helps.

Measures

Task metric: For both binary and multiclass classification, we will report validation accuracy, precision, recall scores, and F1-scores classification metrics on the validation and test sets as used in [2,7]. Class performance will be assessed with confusion matrices and recall scores. After selection, the same metrics will be used on the test set for final performance. **Useful outcome**: We aim to prove that augmentation improves the generalizability of our classifier, and our chosen task metrics will be used to assess overall and class-specific improvement. **Failure/Diagnostic**: We will monitor training and validation loss and accuracy to identify possible overfitting, underfitting, or instability. The confusion matrices and recall scores will be useful to identify any class imbalances and tumor categories that are frequently misclassified. We will also compare accuracy across axial, coronal, and sagittal slices to see if the model struggles with any one particular view. **End to end check:** Before full training, we run the whole pipeline on a small sample of about 100 images per class or less. We will verify the images load correctly, expected shape at each step, proprocessing is performed as expected, the labels match the intended class mapping (e.g., glioma = 0), and the model can train and make predictions without errors. The evaluation code should also calculate the metrics correctly.

Fair test: All model comparisons and metrics will use the same train/val/test splits. Training settings, hyperparameters, experimental variables (e.g. augmentation) only affect the training and validation sets. The provided 1,000-image test set is kept untouched until the end and is used only once, to report the final metric results of both models.

Feasibility

Resources: The dataset is a ~250 MB zip of 6,000 2D images (about 256×256). **Compute:** Google Colab's free T4 NVIDIA Tesla T4 GPU, with 15 GB of VRAM **Storage:** The zip is kept in Google Drive and unzipped into Colab at the start of each session; model checkpoints are also saved to Drive in case Colab disconnects. **Time budget:** Each run is about 20 epochs at

roughly 1 minute per epoch, so 15–30 minutes per run. Two main runs (with and without augmentation) plus a small end-to-end test come to about 1–2 hours of GPU time, with room for reruns.

Risk and fallback

Main risk: The model overfits, meaning it memorizes the training images instead of learning general patterns, so it does well on training data but worse on new images. **Test and decision date:** By **Nov 11** we train the baseline and compare training and validation metrics. If training accuracy is much higher than validation accuracy (e.g., a gap of more than 5 points), or validation loss starts rising while training loss keeps falling, the model is overfitting. **Fallback:** We add early stopping (stop training when validation accuracy stops improving) and dropout (randomly switching off some connections during training so the model can't memorize). We apply these to both models so the augmentation comparison stays fair.

GitHub Setup and Responsibilities

Repository github.com/potent-yoperator/Nxgen-MRI-Diagnositics (shared, public)

Shorthand	Canvas name	GitHub username	Responsibility and deliverable
P1	Matthew A Lopez Jirau	@potent-yoperator	*Neural model development → both EfficientNet-B0 models with pooling, Dense 100/ReLU, and task-specific output layers*
P2	Rohan Kumaran	@rkumaran6611	*Integration, experiments, and evaluation → runnable end-to-end pipeline with controlled augmentation comparisons, metrics tables, confusion matrices, and final test evaluation*
P3	Inniya Santhanakrishnan	@isanthanakri	*Neural preparation and baseline coordination → reproducible train/validation/test splits and preprocessing/augmentation pipeline*

Work Plan

Target date	Checkpoint	Owner	Completion evidence or dependency
Nov 1	Proposal	P1	*Designed architecture, created repository, and responsibilities*
Nov 11	First baseline	P1	*Usable data and first baseline result*
Nov 18	Neural stage	P2	*Trained/adapted stage and diagnostic result*
Nov 22	Progress	P1	*Report, notebook evidence, findings, next test*
Dec 2	Integration	P2	*Complete pipeline; risk test and fallback decision*
Dec 6	Review ready	P1	*Repository organized, results, and presentation slides*
Dec 7 or 9	Demo	All	*Assigned three-minute presentation: meet and rehearse*
Dec 9	Final	All	*Team package; individual statements and reviews*

References

[1] Fateh et al. *BRISC: Annotated Dataset for Brain Tumor Segmentation and Classification*. 2026. https://doi.org/10.1038/s41597-026-06753-y

[2] Missaoui, R., Del Coco, M., Saadaoui, W., Hechkel, W., Helali, A., Carcagnì, P., and Leo, M. *Brain Tumor Classification Using DINO Features and Lightweight Classifiers*. 2026. https://doi.org/10.3390/electronics15050952

[3] Ibrahim, A. W. *Brain Tumor Detection* [Kaggle notebook]. 2023. https://www.kaggle.com/code/abdallahwagih/brain-tumor-detection

[4] TensorFlow. *TensorFlow Tutorials*. n.d. https://www.tensorflow.org/tutorials

[5] darkhelmet. *TensorFlow Cheat Sheets*. n.d. https://darkhelmet.github.io/cheats/python/tensorflow/

[6] Fateh, A., Rezvani, Y., Moayedi, S., Rezvani, S., Fateh, F., Fateh, M., and Abolghasemi, V. *BRISC 2025* [Dataset]. 2025. https://doi.org/10.34740/KAGGLE/DS/7632487

[7] Fateh, A., Rezvani, Y., Rezvani, S., Fateh, M., & Abolghasemi, V. (2026). *Swin-HAFNet: A hierarchical multi-task transformer for brain tumor segmentation and classification*

[8] OpenAI. MRI tumor classification architecture. AI-assisted diagram created using Codex from project specifications. Nov 1. 2026.